claude-windows / 188dd06f-9c99-487c-aa7d-b0bee83c629c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\188dd06f-9c99-487c-aa7d-b0bee83c629c\subagents\workflows\wf_416cc0e5-a35\agent-a679a8c1b49fcb0fe.jsonl`
- SHA-256: `43ad81b287d0648c8174c04af3c86628e406c3fd04f0e8abf51ded2d2f579660`
- Source modified: `2026-06-22T09:49:47+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf_416cc0e5-a35`
- session_id: `188dd06f-9c99-487c-aa7d-b0bee83c629c`

Transcript

1. user (2026-06-22T09:47:42.443Z)

Review the NIGHTLY REEL-COUNT engine-regression change under C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e (a NEW, additive PR). It runs the configured WASB pipeline on a GCP GPU VM nightly, asserts reel-count (==) + quality metrics + cost, emails a report, self-deletes. Files: scripts/nightly_reelcount.py, scripts/nightly_email.py, deploy/gcp/nightly_regression/run_on_vm.sh, deploy/gcp/nightly_regression/launch.sh, deploy/gcp/nightly_regression/register_scheduler.sh, deploy/gcp/nightly_regression/Dockerfile.launcher, eval_baselines/reelcount_manifest.json. Read them under that absolute path. Cite file:line. Report only REAL defects (correctness/safety/security that would bite in production or a regression that escapes), not style. Return raw structured findings.

ADVERSARIALLY VERIFY this finding ? confirm with concrete code evidence under C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e or REFUTE it (default real=false if the evidence isn't clearly there). Finding:


```
{
 "title": "No audit logging of SMTP login attempts or failures",
 "file": "scripts/nightly_email.py",
 "line": "79-91",
 "severity": "low",
 "detail": "Successful SMTP sends are logged (line 90: `log.info('emailed report to %s', to_addr)`), but failed login attempts only log a generic warning (line 81). If an attacker exhausts SMTP login retries or performs credential-stuffing, there is no detailed audit trail. This is low-severity because the credentials are app-password (not the main Gmail password) and the account is monitored by the owner.",
 "fix": "Log SMTP connection/auth failures with details (host, port, user) but never the password. Current behavior is acceptable for a tripwire system."
}
```


2. assistant (2026-06-22T09:47:44.696Z)

I'll systematically examine the nightly regression engine code to verify this audit logging finding. Let me start by reading the relevant files.

3. user (2026-06-22T09:47:45.258Z)

```
1       """Email the nightly regression report ? stdlib SMTP, creds from env or GCP Secret
Manager.
2
3       Used by ``scripts/nightly_reelcount.py --email`` (and reusable by any nightly).
Deliberately
4       dependency-light (stdlib ``smtplib`` + ``email.message``) and **graceful**: if no SMTP
password can
5       be resolved it logs + returns ``False`` (the report is still written/printed ? a broken
mailer must
```

```
never mask a regression).

Credential resolution (first hit wins), so it works on a GCP VM, locally, or in tests:
  ? Recipient:  ``--email-to`` arg  ?  ``NIGHTLY_EMAIL_TO``  ?  ``avi.dullu@gmail.com``
  ? SMTP host:  ``NIGHTLY_SMTP_HOST``  (default ``smtp.gmail.com``)   · port ``NIGHTLY_SMTP_PORT`` (587, STARTTLS)
  ? SMTP user:  ``NIGHTLY_SMTP_USER``  (default = recipient)
  ? SMTP pass:  ``NIGHTLY_SMTP_PASS``  ?  Secret Manager ``NIGHTLY_SMTP_SECRET``
                (``projects/<p>/secrets/<name>/versions/latest``; via the google-cloud-secret-manager
                 lib if installed, else the ``gcloud`` CLI). A Gmail **app password** is the simplest.

Set the secret once (owner):  echo -n '<app-password>' | gcloud secrets create nightly-smtp-pass --data-file=-
and on the VM:  export NIGHTLY_SMTP_SECRET=projects/<proj>/secrets/nightly-smtp-pass/versions/latest
                export NIGHTLY_SMTP_USER=<your-gmail> NIGHTLY_EMAIL_TO=avi.dullu@gmail.com
"""
from __future__ import annotations

import logging
import os
from email.message import EmailMessage
from typing import Optional

log = logging.getLogger("nightly_email")

DEFAULT_TO = "avi.dullu@gmail.com"
DEFAULT_HOST = "smtp.gmail.com"
DEFAULT_PORT = 587


def build_message(subject: str, body_md: str, from_addr: str, to_addr: str) -> EmailMessage:
    """A text/plain email carrying the markdown report (renders fine in any client). Pure ? testable."""
    msg = EmailMessage()
    msg["Subject"] = subject
    msg["From"] = from_addr
    msg["To"] = to_addr
    msg.set_content(body_md)
    return msg


def resolve_secret(env: Optional[dict] = None) -> Optional[str]:
    """Resolve the SMTP password from env or GCP Secret Manager. Returns None if unresolvable."""
    env = env if env is not None else os.environ
    direct = env.get("NIGHTLY_SMTP_PASS")
    if direct:
        return direct
    secret_ref = env.get("NIGHTLY_SMTP_SECRET")
    if not secret_ref:
        return None
    # Prefer the client lib; fall back to the gcloud CLI (no extra dependency required).
    try:
        from google.cloud import secretmanager  # type: ignore
        client = secretmanager.SecretManagerServiceClient()
        return client.access_secret_version(name=secret_ref).payload.data.decode("utf-8")
    except Exception:  # noqa: BLE001 ? lib absent or auth issue ? try the CLI
        pass
    try:
```

```
        import subprocess
        name = secret_ref.split("/secrets/", 1)[1].split("/", 1)[0] if "/secrets/" in secret_ref else secret_ref
        out = subprocess.run(["gcloud", "secrets", "versions", "access", "latest", "--secret", name],
                             capture_output=True, text=True, timeout=30)
        return out.stdout.strip() if out.returncode == 0 and out.stdout.strip() else None
    except Exception as e:  # noqa: BLE001
        log.warning("secret-manager resolution failed: %s", e)
        return None


def send_report(subject: str, body_md: str, to: Optional[str] = None,
                env: Optional[dict] = None) -> bool:
    """Send the report. Returns True if sent, False if skipped (no creds) ? never raises on missing creds."""
    env = env if env is not None else os.environ
    to_addr = to or env.get("NIGHTLY_EMAIL_TO") or DEFAULT_TO
    host = env.get("NIGHTLY_SMTP_HOST", DEFAULT_HOST)
    port = int(env.get("NIGHTLY_SMTP_PORT", DEFAULT_PORT))
    user = env.get("NIGHTLY_SMTP_USER") or to_addr
    password = resolve_secret(env)
    if not password:
        log.warning("no SMTP password (set NIGHTLY_SMTP_PASS or NIGHTLY_SMTP_SECRET) ? email skipped.")
        return False

    import smtplib
    msg = build_message(subject, body_md, from_addr=user, to_addr=to_addr)
    with smtplib.SMTP(host, port, timeout=60) as s:
        s.starttls()
        s.login(user, password)
        s.send_message(msg)
    log.info("emailed report to %s", to_addr)
    return True

```

4. user (2026-06-22T09:47:46.351Z)

```
"""Nightly END-TO-END engine regression ? reel-count + quality + cost (the GCP-VM nightly).

Runs the **configured + checkpointed** pipeline (the real WASB GPU detector, as a deployment serves
it) end-to-end on 2-3 short owned golden videos and asserts the **number of reels does not change**
vs a frozen baseline. Unlike ``scripts/nightly_regression.py`` (PR #202 ? a CPU re-score of CACHED
trajectories, the windowing/scorer only), this one runs the *full* pipeline on the *real model*, so it
needs a GPU ? hence it runs on an ephemeral GCP GPU VM (``deploy/gcp/nightly_regression/``), emails the
owner the report, and self-deletes. It is **default-additive**: a tripwire, not the promotion gate.

What it asserts / reports, per the owner's ask ("quality metrics + regression report + cost"):
  ? REEL COUNT per video ? ``len(play_segments)`` after segmentation; the detected highlight clips.
    Pure-CV / WASB detection is deterministic for a fixed (video, config, weights), so this is asserted
    **EXACT** (with a re-run-once guard for the one known flake: a transient DB-insert-None, see §guard).
  ? QUALITY METRICS per video (when golden labels are provided) ? the R2 tolerance-match
```

```
block from
    ``backend.eval.rally_seg_eval`` (tol_f1 / f1@0.5 / f1@0.25), compared with a small tolerance.
  ? COST of the run ? VM uptime (or processing wall) × machine $/hr via ``backend.billing`` ? USD + INR.

Extensible pattern (add a video = one manifest row + re-freeze):
  ``eval_baselines/reelcount_manifest.json``  ? the videos (gs:// / local / gdrive:// URIs) + segmenter
  ``eval_baselines/reelcount_baseline.json``  ? the frozen expected counts (regenerated, never hand-edited)

Usage
-----
Freeze the baseline from the current configured+checkpointed engine (after an intended change, or first run):
    python scripts/nightly_reelcount.py --update-baseline

Nightly check (exit 1 on regression; writes output/nightly_reelcount/<date>.md; emails if --email):
    python scripts/nightly_reelcount.py --email

Exit codes: 0 pass · 1 regression (reel-count changed / a video errored / quality dropped) · 2 operational
(manifest/videos missing, nothing ran) · 3 no baseline · 4 stale (corpus/segmenter changed ? re-freeze).
"""
from __future__ import annotations

import argparse
import datetime as _dt
import json
import os
import sys
from dataclasses import dataclass, field
from typing import Any, Dict, List, Optional, Tuple

REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
if REPO_ROOT not in sys.path:
    sys.path.insert(0, REPO_ROOT)

# billing is pure (typing only) ? safe to import at module top.
from backend import billing  # noqa: E402

DEFAULT_MANIFEST = os.path.join(REPO_ROOT, "eval_baselines", "reelcount_manifest.json")
DEFAULT_BASELINE = os.path.join(REPO_ROOT, "eval_baselines", "reelcount_baseline.json")
DEFAULT_REPORT_DIR = os.path.join(REPO_ROOT, "output", "nightly_reelcount")

#: Quality metrics we report + (when labels exist) gate ? higher is better, small tolerance.
QUALITY_HIGHER: Tuple[str, ...] = ("tol_f1", "f1@0.5", "f1@0.25")
DEFAULT_QUALITY_TOL = 0.02     # quality is float/labeled ? a looser tripwire than the exact reel-count
DEFAULT_FX_INR_PER_USD = 83.0
#: Fallback machine price if the manifest doesn't pin one (T4 on n1-standard-8, asia-south1, ~mid-2026).
DEFAULT_MACHINE_USD_PER_HR = 0.35


@dataclass(frozen=True)
class Tolerances:
    quality_tol: float = DEFAULT_QUALITY_TOL

    def to_dict(self) -> Dict[str, Any]:
        return {"quality_tol": self.quality_tol}
```

```
# --------------------------------------------------------------------------- #
# Pure cost math (wraps backend.billing ? no IO).
# --------------------------------------------------------------------------- #
def cost_report(billed_seconds: float, machine_usd_per_hr: float,
                fx_inr_per_usd: float, gpu_seconds: float = 0.0) -> Dict[str, Any]:
    """Run cost from billed wall-seconds × machine rate ? USD + INR (``backend.billing`` arithmetic).

    ``billed_seconds`` is the VM uptime (boot?delete) when the orchestrator passes ``--vm-uptime-seconds``;
    otherwise it's the in-process pipeline wall time (a lower bound ? the VM also pays for boot/install)."""
    rate_per_sec = float(machine_usd_per_hr) / 3600.0
    usage = {billing.WALL_SECONDS: float(billed_seconds)}
    rates = {billing.WALL_SECONDS: rate_per_sec}
    usd, breakdown = billing.compute_cost_usd(usage, rates)
    return {
        "billed_seconds": round(float(billed_seconds), 1),
        "gpu_seconds": round(float(gpu_seconds), 1),
        "machine_usd_per_hr": float(machine_usd_per_hr),
        "usd": usd,
        "inr": billing.to_inr(usd, fx_inr_per_usd),
        "breakdown": breakdown,
    }


# --------------------------------------------------------------------------- #
# Pure summary + comparison core (no IO ? unit-testable with plain dicts).
# --------------------------------------------------------------------------- #
def summarize_results(run_results: List[Dict[str, Any]], segmenter: str,
                      cost: Dict[str, Any]) -> Dict[str, Any]:
    """Compact, comparable snapshot from per-video run results.

    Each run result: ``{name, reel_count, ok, error, quality: {tol_f1,...}|None, wall_seconds}``.
    A video that errored keeps ``reel_count=None`` + ``ok=False`` (so the diff flags it, never hides it)."""
    per_video: Dict[str, Any] = {}
    for r in run_results:
        per_video[r["name"]] = {
            "reel_count": r.get("reel_count"),
            "ok": bool(r.get("ok", False)),
            "error": r.get("error"),
            "quality": r.get("quality"),
        }
    return {"segmenter": segmenter, "cost": cost, "per_video": per_video,
            "n": len(per_video)}


@dataclass
class Finding:
    video: str
    metric: str                 # "reel_count" | "error" | a quality key | "segmenter" | "corpus"
    kind: str                   # "regression" | "improvement" | "stale"
    baseline: Optional[float] = None
    current: Optional[float] = None
    detail: str = ""

    def message(self) -> str:
        if self.kind == "stale":
            return f"[STALE] {self.metric}: {self.detail}"
        if self.metric == "error":
```

```
            return f"[REGRESSION] {self.video}: pipeline ERROR ? {self.detail}"
        b = "?" if self.baseline is None else f"{self.baseline:g}"
        c = "?" if self.current is None else f"{self.current:g}"
        tag = "REGRESSION" if self.kind == "regression" else "improvement"
        return f"[{tag}] {self.video}/{self.metric}: {b} ? {c}"


@dataclass
class NightlyResult:
    findings: List[Finding] = field(default_factory=list)
    added: List[str] = field(default_factory=list)
    removed: List[str] = field(default_factory=list)

    @property
    def regressions(self) -> List[Finding]:
        return [f for f in self.findings if f.kind == "regression"]

    @property
    def improvements(self) -> List[Finding]:
        return [f for f in self.findings if f.kind == "improvement"]

    @property
    def stale(self) -> List[Finding]:
        return [f for f in self.findings if f.kind == "stale"]

    def overall_status(self) -> str:
        if self.regressions:
            return "REGRESSION"
        if self.stale:
            return "STALE"
        return "PASS"

    def exit_code(self) -> int:
        if self.regressions:
            return 1
        if self.stale:
            return 4
        return 0


def compare(baseline: Dict[str, Any], current: Dict[str, Any], tols: Tolerances) -> NightlyResult:
    """Diff current run vs frozen baseline.

    * Segmenter changed ? STALE (numbers not comparable; re-freeze).
    * Corpus (video set) changed ? STALE + report added/removed; still diff the INTERSECTION.
    * Per shared video: reel_count EXACT (any change = regression); a pipeline ERROR = regression;
      quality metrics drop beyond ``quality_tol`` = regression (rise = improvement).
    """
    res = NightlyResult()
    if baseline.get("segmenter") != current.get("segmenter"):
        res.findings.append(Finding("", "segmenter", "stale",
                                    detail=f"{baseline.get('segmenter')} ? {current.get('segmenter')}"))
        return res

    b_pv, c_pv = baseline.get("per_video", {}), current.get("per_video", {})
    res.added = sorted(set(c_pv) - set(b_pv))
    res.removed = sorted(set(b_pv) - set(c_pv))
    if res.added or res.removed:
        res.findings.append(Finding("", "corpus", "stale",
                                    detail=f"added={res.added} removed={res.removed}"))

```

```python
    for v in sorted(set(b_pv) & set(c_pv)):
        b, c = b_pv[v], c_pv[v]
        # 1) pipeline must still succeed
        if not c.get("ok", False) or c.get("reel_count") is None:
            res.findings.append(Finding(v, "error", "regression",
                                        detail=str(c.get("error") or "no reel_count produced")))
            continue
        # 2) reel count ? EXACT
        bc, cc = b.get("reel_count"), c.get("reel_count")
        if bc is not None and cc is not None and cc != bc:
            res.findings.append(Finding(v, "reel_count", "regression", float(bc), float(cc)))
        # 3) quality metrics ? tolerance (only if both sides have them)
        bq, cq = b.get("quality") or {}, c.get("quality") or {}
        for m in QUALITY_HIGHER:
            bv, cv = bq.get(m), cq.get(m)
            if bv is None or cv is None:
                continue
            if cv < bv - tols.quality_tol:
                res.findings.append(Finding(v, m, "regression", bv, cv))
            elif cv > bv + tols.quality_tol:
                res.findings.append(Finding(v, m, "improvement", bv, cv))
    return res


# --------------------------------------------------------------------------- #
# Report formatting (pure).
# --------------------------------------------------------------------------- #
def _q(qd: Optional[Dict[str, Any]], key: str) -> str:
    if not qd or qd.get(key) is None:
        return "?"
    return f"{qd[key]:.3f}"


def format_report(baseline: Dict[str, Any], current: Dict[str, Any],
                  result: NightlyResult, date: str, head: str) -> str:
    status = result.overall_status()
    badge = {"PASS": "? PASS", "REGRESSION": "? REGRESSION",
             "STALE": "? STALE (re-freeze the baseline)"}[status]
    cost = current.get("cost", {})
    L: List[str] = [
        f"# Nightly engine regression (reel-count + quality + cost) ? {date}",
        "",
        f"**Status: {badge}**  ·  exit code {result.exit_code()}  ·  segmenter `{current.get('segmenter')}`",
        "",
        f"- HEAD `{head}`  ·  baseline `{baseline.get('git_commit', '?')}` (frozen {baseline.get('generated_at', '?')})",
        f"- **Run cost: ${cost.get('usd', 0):.4f} (?{cost.get('inr', 0):.2f})**  ·  "
        f"billed {cost.get('billed_seconds', 0):.0f}s @ ${cost.get('machine_usd_per_hr', 0)}/hr",
        "",
    ]
    if result.regressions:
        L += ["## ? Regressions", ""] + [f"- {f.message()}" for f in result.regressions] + [""]
    if result.stale:
        L += ["## ? Stale / not comparable", ""] + [f"- {f.message()}" for f in result.stale]
        L += ["- Action: `python scripts/nightly_reelcount.py --update-baseline` (after confirming the change is intended).", ""]

    # Per-video table.
    b_pv, c_pv = baseline.get("per_video", {}), current.get("per_video", {})
```

```
    L += ["## Per-video", "",
          "| video | reels (base?now) | tol_f1 (base?now) | f1@0.5 | status |",
          "|---|---|---|---|---|"]
    for v in sorted(set(b_pv) | set(c_pv)):
        b, c = b_pv.get(v, {}), c_pv.get(v, {})
        bc = b.get("reel_count")
        cc = c.get("reel_count")
        rc = f"{'?' if bc is None else bc}?{'ERROR' if not c.get('ok', True) else ('?' if cc is None else cc)}"
        bq, cq = b.get("quality"), c.get("quality")
        tf = f"{_q(bq, 'tol_f1')}?{_q(cq, 'tol_f1')}"
        f5 = f"{_q(bq, 'f1@0.5')}?{_q(cq, 'f1@0.5')}"
        vstatus = "?" if any(f.video == v and f.kind == "regression" for f in result.findings) else "?"
        L.append(f"| {v} | {rc} | {tf} | {f5} | {vstatus} |")
    L.append("")

    if result.improvements:
        L += ["_Improvements over baseline (re-freeze to ratchet up):_"] \
            + [f"- {f.message()}" for f in result.improvements] + [""]
    L += ["---",
          "_Full configured+checkpointed pipeline on the real model (GCP GPU VM). Tripwire only ? does "
          "not change the promotion gate. Complements PR #202's cached-trajectory CPU re-score._"]
    return "\n".join(L)


# --------------------------------------------------------------------------- #
# IO shell ? running the configured pipeline per video (needs backend + GPU + video).
# --------------------------------------------------------------------------- #
def _load_gts(labels_ref: Optional[str]) -> Optional[List[Tuple[float, float]]]:
    """Load ground-truth rally intervals from a golden-features JSON (``gts`` key) or a rallies CSV
    (``start,end`` columns / pairs). Returns None when no labels are configured."""
    if not labels_ref:
        return None
    from backend.storage import fetch
    local = fetch(labels_ref)
    if local.endswith(".json"):
        with open(local, encoding="utf-8") as fh:
            data = json.load(fh)
        return [(float(s), float(e)) for s, e in (data.get("gts") or [])]
    # CSV: header start,end (or first two numeric columns)
    import csv
    out: List[Tuple[float, float]] = []
    with open(local, encoding="utf-8") as fh:
        for row in csv.reader(fh):
            try:
                out.append((float(row[0]), float(row[1])))
            except (ValueError, IndexError):
                continue
    return out


def _segments_to_intervals(segs: List[Dict[str, Any]]) -> List[Tuple[float, float]]:
    """``(start, end)`` pairs from segmenter result dicts, tolerant of the two key conventions in the
    codebase (``start_time``/``end_time`` ? motion/DB shape ? or ``start``/``end``). Used only for the
    optional quality metrics; the reel COUNT is ``len(segs)`` regardless, so a key mismatch degrades
    quality scoring gracefully (skipped) without ever miscounting reels."""
    out: List[Tuple[float, float]] = []
    for s in segs:
```

```
        start = s.get("start_time", s.get("start"))
        end = s.get("end_time", s.get("end"))
        if start is not None and end is not None:
            out.append((float(start), float(end)))
    return out


def run_one_video(video: Dict[str, Any], segmenter: str, config: Any,
                  rerun_on_mismatch: bool = True,
                  baseline_count: Optional[int] = None) -> Dict[str, Any]:
    """Run the configured pipeline on one video ? reel count (+ quality vs labels). Lazy backend imports.

    The re-run-once guard (the one known non-determinism ? a transient ``add_play_segment``?None drop,
    database.py): if the count != the frozen baseline, re-process the video ONCE; a transient drop won't
    reproduce, a real change will."""
    import random
    import time

    from backend.eval import rally_seg_eval
    from backend.infrastructure.database import Database
    from backend.pipeline.segmenters import segmenter_registry
    from backend.results import Failure
    from backend.storage import fetch
    from backend.utils.hashing import compute_video_id

    name = video["name"]
    try:
        local = fetch(video["uri"])
    except Exception as e:  # noqa: BLE001 ? surface, never hide
        return {"name": name, "reel_count": None, "ok": False, "error": f"fetch failed: {e}",
                "quality": None, "wall_seconds": 0.0}

    gts = _load_gts(video.get("labels_uri"))
    seg_cls = segmenter_registry.get(segmenter)
    if not seg_cls:
        return {"name": name, "reel_count": None, "ok": False,
                "error": f"segmenter '{segmenter}' not registered", "quality": None, "wall_seconds": 0.0}

    def _process(db_path: str) -> Tuple[Optional[int], Optional[List[Tuple[float, float]]], float, Optional[str]]:
        random.seed(0)  # close motion.py's unseeded metadata RNG so secondary fields are reproducible too
        db = Database(db_path)
        video_id = compute_video_id(local)
        t0 = time.monotonic()
        result = seg_cls(db, config).process_video(video_id, local)
        wall = time.monotonic() - t0
        if isinstance(result, Failure):
            return None, None, wall, getattr(result, "message", "segmenter failed")
        segs = result.value if hasattr(result, "value") else result
        intervals = _segments_to_intervals(segs)
        return len(segs), intervals, wall, None

    import tempfile
    total_wall = 0.0
    with tempfile.TemporaryDirectory(prefix="nightly_reel_") as td:
        count, intervals, wall, err = _process(os.path.join(td, "run1.db"))
        total_wall += wall
        if err is not None:
            return {"name": name, "reel_count": None, "ok": False, "error": err,
```

```
                    "quality": None, "wall_seconds": round(total_wall, 2)}
        # re-run-once guard for the transient DB-drop flake
        if rerun_on_mismatch and baseline_count is not None and count != baseline_count:
            count2, intervals2, wall2, err2 = _process(os.path.join(td, "run2.db"))
            total_wall += wall2
            if err2 is None and count2 is not None:
                count, intervals = count2, intervals2  # the reproduced (or corrected) value

        quality = None
        if gts and intervals is not None:
            row = rally_seg_eval.score_intervals(intervals, gts)
            quality = {k: row[k] for k in ("tol_f1", "f1@0.5", "f1@0.25", "tol_precision", "tol_recall")
                       if k in row}

    return {"name": name, "reel_count": count, "ok": True, "error": None,
            "quality": quality, "wall_seconds": round(total_wall, 2)}


def load_manifest(path: str) -> Dict[str, Any]:
    with open(path, encoding="utf-8") as fh:
        return json.load(fh)


def run_corpus(manifest: Dict[str, Any], baseline: Optional[Dict[str, Any]],
               rerun_on_mismatch: bool = True) -> Tuple[List[Dict[str, Any]], float]:
    """Run every manifest video through the configured pipeline. Returns (run_results, total_wall_s)."""
    from backend.config import load_config

    config = load_config()
    # apply manifest config overrides onto the typed config (dotted keys)
    for dotted, val in (manifest.get("config_overrides") or {}).items():
        _set_dotted(config, dotted, val)
    segmenter = manifest.get("segmenter") or getattr(getattr(config, "indexing", None), "default_segmenter", "motion")
    b_pv = (baseline or {}).get("per_video", {})
    results: List[Dict[str, Any]] = []
    total_wall = 0.0
    for v in manifest.get("videos", []):
        base_count = (b_pv.get(v["name"]) or {}).get("reel_count")
        r = run_one_video(v, segmenter, config, rerun_on_mismatch, base_count)
        total_wall += float(r.get("wall_seconds") or 0.0)
        results.append(r)
    return results, total_wall


def _set_dotted(config: Any, dotted: str, value: Any) -> None:
    """Best-effort dotted-path set onto a pydantic config (e.g. 'indexing.motion_threshold')."""
    parts = dotted.split(".")
    obj = config
    for p in parts[:-1]:
        obj = getattr(obj, p, None)
        if obj is None:
            return
    try:
        setattr(obj, parts[-1], value)
    except Exception:  # noqa: BLE001 ? overrides are best-effort; a bad key shouldn't crash the run
        pass


def _git_head() -> str:
```

```
    import subprocess
    try:
        out = subprocess.run(["git", "-C", REPO_ROOT, "rev-parse", "--short", "HEAD"],
                             capture_output=True, text=True, timeout=10)
        return out.stdout.strip() or "?"
    except Exception:  # noqa: BLE001
        return "?"


def load_baseline(path: str) -> Optional[Dict[str, Any]]:
    if not os.path.exists(path):
        return None
    with open(path, encoding="utf-8") as fh:
        return json.load(fh)


def save_baseline(path: str, snapshot: Dict[str, Any]) -> None:
    out = dict(snapshot)
    out["generated_at"] = _dt.date.today().isoformat()
    out["git_commit"] = _git_head()
    out["_doc"] = ("Frozen nightly reel-count baseline. Regenerate with `python scripts/nightly_reelcount.py "
                   "--update-baseline` after an INTENDED engine change or when adding a video. Tied to the "
                   "configured+checkpointed engine + the manifest corpus.")
    out.pop("cost", None)  # cost is per-run, not part of the frozen baseline
    os.makedirs(os.path.dirname(path), exist_ok=True)
    tmp = path + ".tmp"
    with open(tmp, "w", encoding="utf-8") as fh:
        json.dump(out, fh, indent=2, sort_keys=True)
        fh.write("\n")
    os.replace(tmp, path)


def build_parser() -> argparse.ArgumentParser:
    p = argparse.ArgumentParser(description="Nightly E2E engine regression: reel-count + quality + cost.")
    p.add_argument("--manifest", default=DEFAULT_MANIFEST)
    p.add_argument("--baseline", default=DEFAULT_BASELINE)
    p.add_argument("--report-dir", default=DEFAULT_REPORT_DIR)
    p.add_argument("--quality-tol", type=float, default=DEFAULT_QUALITY_TOL)
    p.add_argument("--vm-uptime-seconds", type=float, default=None,
                   help="authoritative VM uptime (boot?delete) for the cost line; "
                        "defaults to the in-process pipeline wall time (a lower bound)")
    p.add_argument("--no-rerun-guard", action="store_true",
                   help="disable the re-run-once-on-mismatch guard (the DB-drop flake mitigation)")
    p.add_argument("--update-baseline", action="store_true",
                   help="freeze the current reel counts + quality as the new baseline instead of checking")
    p.add_argument("--email", action="store_true", help="email the report (see scripts/nightly_email.py)")
    p.add_argument("--email-to", default=None, help="override the report recipient")
    p.add_argument("--date", default=None, help="override the report date (YYYY-MM-DD; for tests)")
    p.add_argument("--no-report", action="store_true", help="skip writing the dated report file")
    return p


def main(argv: Optional[List[str]] = None) -> int:
    import logging
    logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
    log = logging.getLogger("nightly_reelcount")
    args = build_parser().parse_args(argv)
```

```
    tols = Tolerances(quality_tol=args.quality_tol)
    date = args.date or _dt.date.today().isoformat()

    if not os.path.exists(args.manifest):
        log.error("manifest not found: %s", args.manifest)
        return 2
    manifest = load_manifest(args.manifest)
    baseline = load_baseline(args.baseline)

    try:
        run_results, total_wall = run_corpus(manifest, baseline, rerun_on_mismatch=not args.no_rerun_guard)
    except Exception as e:  # noqa: BLE001 ? operational failure (deps/config) ? exit 2, never a false "pass"
        log.error("run failed: %s", e)
        return 2
    if not run_results:
        log.error("no videos in manifest %s", args.manifest)
        return 2

    billed = args.vm_uptime_seconds if args.vm_uptime_seconds is not None else total_wall
    cost = cost_report(
        billed_seconds=billed,
        machine_usd_per_hr=float(manifest.get("machine_usd_per_hr", DEFAULT_MACHINE_USD_PER_HR)),
        fx_inr_per_usd=float(manifest.get("fx_inr_per_usd", DEFAULT_FX_INR_PER_USD)),
        gpu_seconds=total_wall,
    )
    segmenter = manifest.get("segmenter", "motion")
    snapshot = summarize_results(run_results, segmenter, cost)
    snapshot["git_commit"] = _git_head()
    snapshot["generated_at"] = date

    if args.update_baseline:
        save_baseline(args.baseline, snapshot)
        log.info("baseline frozen ? %s", args.baseline)
        for r in run_results:
            log.info("  %s: reels=%s ok=%s", r["name"], r.get("reel_count"), r.get("ok"))
        return 0

    if baseline is None:
        log.error("no baseline at %s ? freeze one with --update-baseline (on the configured engine).", args.baseline)
        return 3

    result = compare(baseline, snapshot, tols)
    report = format_report(baseline, snapshot, result, date, snapshot["git_commit"])

    if not args.no_report:
        os.makedirs(args.report_dir, exist_ok=True)
        md_path = os.path.join(args.report_dir, f"{date}.md")
        with open(md_path, "w", encoding="utf-8") as fh:
            fh.write(report + "\n")
        with open(os.path.join(args.report_dir, f"{date}.json"), "w", encoding="utf-8") as fh:
            json.dump({"date": date, "status": result.overall_status(), "exit_code": result.exit_code(),
                       "snapshot": snapshot, "regressions": [f.message() for f in result.regressions]},
                      fh, indent=2, sort_keys=True)
        log.info("report ? %s", md_path)

    print(report)
```

```python

    if args.email:
        try:
            from scripts.nightly_email import send_report
            subject = f"[KhelSutra nightly] {result.overall_status()} ? reel-count engine regression {date}"
            sent = send_report(subject, report, to=args.email_to)
            log.info("email %s", "sent" if sent else "skipped (no creds ? report still written)")
        except Exception as e:  # noqa: BLE001 ? a broken mailer must NOT mask the regression verdict
            log.error("email failed (report still written): %s", e)

    code = result.exit_code()
    if code == 1:
        log.error("REEL-COUNT REGRESSION (%d) ? see %s", len(result.regressions), args.report_dir)
    elif code == 4:
        log.warning("baseline STALE ? corpus/segmenter changed; re-freeze with --update-baseline")
    else:
        log.info("nightly reel-count check PASSED  (cost $%.4f / ?%.2f)", cost["usd"], cost["inr"])
    return code


if __name__ == "__main__":
    raise SystemExit(main())

```

5. user (2026-06-22T09:47:46.909Z)

```bash
#!/usr/bin/env bash
# Startup-script for the NIGHTLY engine-regression VM. Runs UNATTENDED on boot, then DELETES the VM
# (cost rule: a running GPU VM is the only real cost). Survives SSH drops (it's a startup-script, not
# an interactive session). Reads its config from instance metadata (set by launch.sh).
#
# Flow:  Ops Agent  ?  pull repo @ HEAD from GCS  ?  build venv + WASB env  ?  stage weights  ?
#         run scripts/nightly_reelcount.py on the gs:// golden clips (emails the report) ?
#         upload report to GCS  ?  SELF-DELETE (even on failure, via the EXIT trap).
#
# Metadata keys (launch.sh sets these): repo-tgz-uri, git-sha, gcs-bucket, report-prefix,
#   weights-uri, email (true/false). NOT verified in CI ? dry-run via launch.sh on a real VM first.
set -uo pipefail
exec > >(tee -a /var/log/nightly-regression.log) 2>&1
echo "===== nightly-regression startup $(date -u +%FT%TZ) ====="
BOOT_EPOCH=$(date +%s)

md() { curl -s -H "Metadata-Flavor: Google" "http://metadata.google.internal/computeMetadata/v1/$1"; }
NAME=$(md instance/name)
ZONE=$(md instance/zone | awk -F/ '{print $NF}')
REPO_TGZ=$(md instance/attributes/repo-tgz-uri || true)
GIT_SHA=$(md instance/attributes/git-sha || echo unknown)
BUCKET=$(md instance/attributes/gcs-bucket || true)
REPORT_PREFIX=$(md instance/attributes/report-prefix || echo nightly/reports)
WEIGHTS_URI=$(md instance/attributes/weights-uri || true)
DO_EMAIL=$(md instance/attributes/email || echo true)
SMTP_SECRET=$(md instance/attributes/smtp-secret || true)
```

```
SMTP_USER=$(md instance/attributes/smtp-user || true)
EMAIL_TO=$(md instance/attributes/email-to || echo avi.dullu@gmail.com)
DATE=$(date -u +%F)

# ALWAYS delete the VM on exit (success OR failure) ? never leave a billing GPU VM behind.
cleanup() {
  echo "===== self-delete $NAME ($ZONE) ====="
  gcloud compute instances delete "$NAME" --zone="$ZONE" --quiet || \
    echo "WARN: self-delete failed ? DELETE MANUALLY: gcloud compute instances delete $NAME --zone=$ZONE -q"
}
trap cleanup EXIT

fail_email() {  # best-effort failure alert (the report email won't have fired on an early crash)
  echo "RUN FAILED ? attempting a failure alert"
  if [ "$DO_EMAIL" = "true" ] && [ -d ~/rally ]; then
    cd ~/rally 2>/dev/null && . .venv/bin/activate 2>/dev/null && \
    NIGHTLY_SMTP_SECRET="$SMTP_SECRET" NIGHTLY_SMTP_USER="$SMTP_USER" NIGHTLY_EMAIL_TO="$EMAIL_TO" \
    python - <<PY || true
from scripts.nightly_email import send_report
send_report("[KhelSutra nightly] ERROR ? engine regression VM run failed ($DATE)",
            "The nightly engine-regression run failed on the VM before producing a report.\n"
            "See gs://$BUCKET/$REPORT_PREFIX/$DATE/ and the VM serial log.\nGIT_SHA=$GIT_SHA", to="$EMAIL_TO")
PY
  fi
}
trap 'fail_email' ERR

# 1) Observability (Ops Agent) ? same as create_gpu_vm.sh bakes in.
curl -sSO https://dl.google.com/cloudagents/add-google-cloud-ops-agent-repo.sh && \
  sudo bash add-google-cloud-ops-agent-repo.sh --also-install || echo "WARN: ops-agent install failed (non-fatal)"

# 2) Pull the repo @ HEAD (staged to GCS by launch.sh ? no GitHub auth needed; uses the worker SA).
mkdir -p ~/rally && cd ~/rally
gcloud storage cp "$REPO_TGZ" /tmp/rally.tgz
tar -xzf /tmp/rally.tgz -C ~/rally
echo "$GIT_SHA" > ~/rally/GIT_SHA

# 3) Build the env + native WASB (provider-agnostic DO scripts, reused on GCP per deploy/gcp/README §4).
sudo bash deploy/digitalocean/mount_scratch.sh 2>/dev/null || true
export TMPDIR=/scratch 2>/dev/null || true
./deploy/digitalocean/bootstrap.sh --gpu
. .venv/bin/activate
pip install -e '.[storage-gcs]'
bash deploy/digitalocean/gpu_setup.sh || echo "WARN: gpu_setup (WASB env) returned nonzero ? continuing"

# 4) Stage the WASB checkpoint (gpu_setup does NOT fetch weights ? README §4).
if [ -n "$WEIGHTS_URI" ]; then
  mkdir -p ~/models/WASB-SBDT/pretrained_weights
  gcloud storage cp "$WEIGHTS_URI" ~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar
  export WASB_WEIGHTS=~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar
fi

# 5) Run the regression on the gs:// golden clips in the committed manifest. Cost = VM uptime so far.
```

```
UPTIME=$(( $(date +%s) - BOOT_EPOCH ))
EMAIL_FLAG=""; [ "$DO_EMAIL" = "true" ] && EMAIL_FLAG="--email --email-to $EMAIL_TO"
set +e
NIGHTLY_SMTP_SECRET="$SMTP_SECRET" NIGHTLY_SMTP_USER="$SMTP_USER" NIGHTLY_EMAIL_TO="$EMAIL_TO" \
  python scripts/nightly_reelcount.py --vm-uptime-seconds "$UPTIME" $EMAIL_FLAG
RUN_RC=$?
set -e
echo "runner exit code: $RUN_RC  (0 pass / 1 regression / 4 stale)"

# 6) Upload the dated report to GCS for the record (alongside the email).
if [ -n "$BUCKET" ] && [ -d output/nightly_reelcount ]; then
  gcloud storage cp -r "output/nightly_reelcount/$DATE."* "gs://$BUCKET/$REPORT_PREFIX/$DATE/" || \
    gcloud storage cp -r output/nightly_reelcount "gs://$BUCKET/$REPORT_PREFIX/$DATE/" || true
fi

echo "===== nightly-regression done rc=$RUN_RC $(date -u +%FT%TZ) ====="
trap - ERR   # disable the failure-email trap; the EXIT (self-delete) trap stays
# the EXIT trap deletes the VM

```

6. user (2026-06-22T09:47:47.410Z)

```
#!/usr/bin/env bash
# Launch ONE nightly engine-regression run on an ephemeral GCP GPU VM, then return. The VM runs
# run_on_vm.sh as its startup-script (regression ? email ? SELF-DELETE), so this just stages the repo
# and kicks the VM off. Driver-agnostic: call it from Cloud Scheduler?Cloud Run job, a cron on any
# always-on box, the owner's machine, or a Claude routine. Needs gcloud auth + the repo checkout.
#
#   bash deploy/gcp/nightly_regression/launch.sh                 # L4, stages HEAD, kicks the VM
#   RALLY_GPU=t4 RALLY_REF=$(git rev-parse HEAD) bash deploy/gcp/nightly_regression/launch.sh
#
# ? Run this MANUALLY once as a dry-run (watch the serial log) before wiring a scheduler ? the VM-side
# flow is NOT covered by CI. Tail: gcloud compute instances get-serial-port-output <name> --zone=<z>.
set -uo pipefail

PROJECT="${RALLY_GCP_PROJECT:-gen-lang-client-0306026826}"
BUCKET="${RALLY_BUCKET:-khelsutra-rally-corpus}"
SA="rally-worker@${PROJECT}.iam.gserviceaccount.com"
GPU="${RALLY_GPU:-l4}"
NAME="${RALLY_VM_NAME:-rally-nightly-$(date -u +%Y%m%d)}"
REF="${RALLY_REF:-HEAD}"
WEIGHTS_URI="${RALLY_WEIGHTS_URI:-gs://$BUCKET/weights/wasb_badminton_best.pth.tar}"
REPORT_PREFIX="${RALLY_REPORT_PREFIX:-nightly/reports}"
DO_EMAIL="${RALLY_EMAIL:-true}"
SMTP_SECRET="${NIGHTLY_SMTP_SECRET:-projects/$PROJECT/secrets/nightly-smtp-pass/versions/latest}"
SMTP_USER="${NIGHTLY_SMTP_USER:-}"
EMAIL_TO="${NIGHTLY_EMAIL_TO:-avi.dullu@gmail.com}"
HERE="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
STARTUP="$HERE/run_on_vm.sh"
export CLOUDSDK_CORE_DISABLE_PROMPTS=1

ZONES="${RALLY_GCP_ZONES:-asia-south1-a asia-south1-b asia-south1-c asia-southeast1-a asia-southeast1-b asia-southeast1-c}"
```

```
    if [ "$GPU" = "l4" ]; then MACHINE="g2-standard-4"; ACCEL=(); else MACHINE="n1-standard-8"; ACCEL=(--accelerator=type=nvidia-tesla-t4,count=1); fi

    # 1) Stage the repo @ REF to GCS (no GitHub auth on the VM; the worker SA can read the bucket).
    SHA="$(git rev-parse "$REF")"
    TGZ_URI="gs://$BUCKET/nightly/repo/${SHA}.tgz"
    echo "== staging repo $SHA -> $TGZ_URI =="
    TMP_TGZ="$(mktemp --suffix=.tgz)"
    git archive --format=tar.gz -o "$TMP_TGZ" "$REF"
    gcloud storage cp "$TMP_TGZ" "$TGZ_URI" --project="$PROJECT"
    rm -f "$TMP_TGZ"

    # 2) Zone-sweep create the VM (mirrors create_gpu_vm.sh) with run_on_vm.sh + the config metadata.
    ERR="$(mktemp)"
    for Z in $ZONES; do
      echo "== creating $NAME ($GPU) in $Z =="
      if gcloud compute instances create "$NAME" --project="$PROJECT" --zone="$Z" \
           --machine-type="$MACHINE" "${ACCEL[@]}" \
           --maintenance-policy=TERMINATE --provisioning-model=STANDARD \
           --image-family=common-cu129-ubuntu-2204-nvidia-580 --image-project=deeplearning-platform-release \
           --boot-disk-size=120GB --boot-disk-type=pd-balanced \
           --service-account="$SA" --scopes=cloud-platform \
           --metadata-from-file=startup-script="$STARTUP" \
           --metadata=repo-tgz-uri="$TGZ_URI",git-sha="$SHA",gcs-bucket="$BUCKET",report-prefix="$REPORT_PREFIX",weights-uri="$WEIGHTS_URI",email="$DO_EMAIL",smtp-secret="$SMTP_SECRET",smtp-user="$SMTP_USER",email-to="$EMAIL_TO" \
           2>"$ERR"; then
        echo "? launched $NAME in $Z ? it will run the regression, email $EMAIL_TO, and self-delete."
        echo "   serial log: gcloud compute instances get-serial-port-output $NAME --zone=$Z --project=$PROJECT"
        echo "   if it doesn't self-delete: gcloud compute instances delete $NAME --zone=$Z -q"
        rm -f "$ERR"; exit 0
      fi
      if grep -qiE "STOCKOUT|does not have enough" "$ERR"; then echo "  (no $GPU capacity in $Z ? next zone)"; continue; fi
      echo "  create failed:"; tail -4 "$ERR"; rm -f "$ERR"; exit 1
    done
    echo "? no $GPU capacity in any swept zone ($ZONES). Retry later or RALLY_GPU=t4."
    rm -f "$ERR"; exit 1

```

7. assistant (2026-06-22T09:47:49.799Z)

Now let me read the remaining files:

8. user (2026-06-22T09:47:51.094Z)

```
#!/usr/bin/env bash
# Wire the nightly engine-regression as a CLOUD-NATIVE, scale-to-zero job:
#   Cloud Scheduler (cron)  ?  Cloud Run JOB (the launcher image)  ?  launch.sh
#     ?  ephemeral GPU VM (regression + email + SELF-DELETE).
# The only always-on cost is $0 (Cloud Run jobs + Scheduler are scale-to-zero; the GPU VM is ephemeral).
#
# ? OWNER-RUN, ONCE, after a successful manual `bash deploy/gcp/nightly_regression/launch.sh` dry-run.
# NOT verified in CI (no GCP/GPU here). Prereqs: GPU quota (README §2), the golden clips + weights in
# gs://<bucket>, and the SMTP secret (see README "Enable").
#
```

```
#   bash deploy/gcp/nightly_regression/register_scheduler.sh          # build + deploy + schedule (03:00 IST)
#   RALLY_CRON='0 3 * * *' RALLY_TZ='Asia/Kolkata' bash .../register_scheduler.sh
set -euo pipefail

PROJECT="${RALLY_GCP_PROJECT:-gen-lang-client-0306026826}"
REGION="${RALLY_RUN_REGION:-asia-south1}"
JOB="${RALLY_RUN_JOB:-nightly-regression-launcher}"
SCHED="${RALLY_SCHED_NAME:-nightly-regression}"
CRON="${RALLY_CRON:-0 3 * * *}"          # 03:00 daily
TZ="${RALLY_TZ:-Asia/Kolkata}"
SA="rally-worker@${PROJECT}.iam.gserviceaccount.com"
IMAGE="${RALLY_LAUNCHER_IMAGE:-${REGION}-docker.pkg.dev/${PROJECT}/rally/nightly-launcher:latest}"
REPO_ROOT="$(git rev-parse --show-toplevel)"
export CLOUDSDK_CORE_DISABLE_PROMPTS=1

echo "== enable APIs (run, scheduler, build, artifact registry) =="
gcloud services enable run.googleapis.com cloudscheduler.googleapis.com \
  cloudbuild.googleapis.com artifactregistry.googleapis.com --project="$PROJECT"

echo "== ensure the Artifact Registry repo 'rally' exists =="
gcloud artifacts repositories describe rally --location="$REGION" --project="$PROJECT" >/dev/null 2>&1 || \
  gcloud artifacts repositories create rally --repository-format=docker --location="$REGION" --project="$PROJECT"

echo "== build + push the launcher image from the repo root =="
( cd "$REPO_ROOT" && gcloud builds submit --project="$PROJECT" --tag "$IMAGE" \
    --gcs-source-staging-dir="gs://${RALLY_BUILD_BUCKET:-khelsutra-rally-corpus}/cloudbuild" \
    -f deploy/gcp/nightly_regression/Dockerfile.launcher . )

echo "== create/update the Cloud Run JOB (launcher needs compute-admin to create the GPU VM) =="
gcloud run jobs deploy "$JOB" --project="$PROJECT" --region="$REGION" \
  --image="$IMAGE" --service-account="$SA" --max-retries=0 --task-timeout=900s \
  --set-env-vars="RALLY_GPU=${RALLY_GPU:-l4},RALLY_EMAIL=true,NIGHTLY_EMAIL_TO=${NIGHTLY_EMAIL_TO:-avi.dullu@gmail.com}"

echo "== schedule it (Cloud Scheduler ? Run job admin API) =="
RUN_URI="https://${REGION}-run.googleapis.com/apis/run.googleapis.com/v1/namespaces/${PROJECT}/jobs/${JOB}:run"
gcloud scheduler jobs describe "$SCHED" --location="$REGION" --project="$PROJECT" >/dev/null 2>&1 && \
  gcloud scheduler jobs delete "$SCHED" --location="$REGION" --project="$PROJECT" --quiet || true
gcloud scheduler jobs create http "$SCHED" --project="$PROJECT" --location="$REGION" \
  --schedule="$CRON" --time-zone="$TZ" \
  --uri="$RUN_URI" --http-method=POST \
  --oauth-service-account-email="$SA"

cat <<EOF

? Scheduled '$SCHED' ($CRON $TZ) ? Cloud Run job '$JOB' ? launch.sh ? ephemeral GPU VM.
   IAM the launcher SA needs (grant once if missing):
     roles/compute.instanceAdmin.v1  (create/delete the GPU VM)
     roles/iam.serviceAccountUser    (attach $SA to the VM)
     roles/run.invoker + roles/cloudscheduler.jobRunner (Scheduler ? Run)
   Trigger now to test:  gcloud run jobs execute $JOB --region=$REGION --project=$PROJECT
   Remove:               gcloud scheduler jobs delete $SCHED --location=$REGION; gcloud run jobs delete $JOB --region=$REGION
EOF
```

9. user (2026-06-22T09:47:52.088Z)

```
# Tiny launcher image for the nightly engine-regression Cloud Run JOB. It is NOT the GPU workload ?
# it only stages the repo + creates the ephemeral GPU VM (which self-deletes). Scale-to-zero: the job
# runs for a few seconds nightly, then exits. Base = google/cloud-sdk (gcloud + git preinstalled).
FROM gcr.io/google.com/cloudsdktool/cloud-sdk:slim
WORKDIR /app
# The repo is the build context (run `gcloud builds submit` from the repo root, see register_scheduler.sh).
COPY . /app
ENTRYPOINT ["bash", "deploy/gcp/nightly_regression/launch.sh"]

```

10. user (2026-06-22T09:47:52.849Z)

```
{
  "_doc": "Nightly E2E engine reel-count manifest (scripts/nightly_reelcount.py). The configured+checkpointed pipeline runs on each video on a GCP GPU VM; the reel count + quality metrics are asserted vs eval_baselines/reelcount_baseline.json. ADD A VIDEO = append one row + re-freeze: `python scripts/nightly_reelcount.py --update-baseline` (commit the updated baseline). URIs may be gs:// (recommended, co-located with the VM), gdrive://, or local paths. labels_uri (optional) is a golden-features JSON (gts key) or a start,end CSV ? present => quality metrics (tol_f1/f1@0.5/f1@0.25) are scored too. EDIT the example rows below to your actual short (~5-6 min) OWNED, minors-free clips before enabling.",
  "segmenter": "wasb_hybrid",
  "detector_impl": "native",
  "config_overrides": {
    "indexing.default_segmenter": "wasb_hybrid",
    "indexing.detector_impl": "native"
  },
  "machine_usd_per_hr": 0.70,
  "fx_inr_per_usd": 83.0,
  "videos": [
    {
      "name": "GX030094",
      "uri": "gs://khelsutra-rally-corpus/nightly/videos/GX030094.mp4",
      "labels_uri": "gs://khelsutra-rally-corpus/nightly/labels/GX030094_golden_features.json"
    },
    {
      "name": "Badminton_BXH_2",
      "uri": "gs://khelsutra-rally-corpus/nightly/videos/Badminton_BXH_2.mp4",
      "labels_uri": "gs://khelsutra-rally-corpus/nightly/labels/Badminton_BXH_2_golden_features.json"
    },
    {
      "name": "GX010128",
      "uri": "gs://khelsutra-rally-corpus/nightly/videos/GX010128.mp4",
      "labels_uri": "gs://khelsutra-rally-corpus/nightly/labels/GX010128_golden_features.json"
    }
  ]
}

```

11. assistant (2026-06-22T09:47:55.415Z)

Now let me analyze the code carefully. The finding claims that there's inadequate audit logging of SMTP login attempts/failures. Let me examine the SMTP authentication code in detail:

12. user (2026-06-22T09:47:56.878Z)

```
71      def send_report(subject: str, body_md: str, to: Optional[str] = None,
72                      env: Optional[dict] = None) -> bool:
73          """Send the report. Returns True if sent, False if skipped (no creds) ? never raises
on missing creds."""
74          env = env if env is not None else os.environ
75          to_addr = to or env.get("NIGHTLY_EMAIL_TO") or DEFAULT_TO
76          host = env.get("NIGHTLY_SMTP_HOST", DEFAULT_HOST)
77          port = int(env.get("NIGHTLY_SMTP_PORT", DEFAULT_PORT))
78          user = env.get("NIGHTLY_SMTP_USER") or to_addr
79          password = resolve_secret(env)
80          if not password:
81              log.warning("no SMTP password (set NIGHTLY_SMTP_PASS or NIGHTLY_SMTP_SECRET) ?
email skipped.")
82              return False
83
84          import smtplib
85          msg = build_message(subject, body_md, from_addr=user, to_addr=to_addr)
86          with smtplib.SMTP(host, port, timeout=60) as s:
87              s.starttls()
88              s.login(user, password)
89              s.send_message(msg)
90          log.info("emailed report to %s", to_addr)
91          return True
92
```

13. assistant (2026-06-22T09:47:58.830Z)

Now let me check if there are any exception handlers or try-except blocks around the SMTP login:

14. user (2026-06-22T09:48:00.275Z)

```
54:    try:
58:    except Exception:  # noqa: BLE001 ? lib absent or auth issue ? try the CLI
60:    try:
66:    except Exception as e:  # noqa: BLE001
87:        s.starttls()
88:        s.login(user, password)
```

15. assistant (2026-06-22T09:48:03.088Z)

Critical finding: **lines 86-89 lack any exception handling**. Let me verify this by looking at the full context again and checking if there's error handling at a higher level:

16. user (2026-06-22T09:48:04.256Z)

Syntax OK

17. assistant (2026-06-22T09:48:06.349Z)

Now let me trace how this function is called to see if there's error handling at the call site:

18. user (2026-06-22T09:48:07.788Z)

```
deploy\gcp\nightly_regression\run_on_vm.sh:46:send_report("[KhelSutra nightly] ERROR ? engine
regression VM run failed ($DATE)",
scripts\nightly_reelcount.py:537:            sent = send_report(subject, report,
to=args.email_to)
scripts\nightly_email.py:71:def send_report(subject: str, body_md: str, to: Optional[str] =
None,
tests\test_nightly_reelcount.py:199:    assert ne.send_report("s", "b", to="x@y.com",
env={"NIGHTLY_EMAIL_TO": "x@y.com"}) is False
```

19. assistant (2026-06-22T09:48:09.736Z)

Let me check the call site in nightly_reelcount.py to see if there's error handling:

20. user (2026-06-22T09:48:10.748Z)

```
if args.email:
    try:
        from scripts.nightly_email import send_report
        subject = f"[KhelSutra nightly] {result.overall_status()} ? reel-count engine regression {date}"
        sent = send_report(subject, report, to=args.email_to)
        log.info("email %s", "sent" if sent else "skipped (no creds ? report still written)")
    except Exception as e:  # noqa: BLE001 ? a broken mailer must NOT mask the regression verdict
        log.error("email failed (report still written): %s", e)

code = result.exit_code()
```

21. assistant (2026-06-22T09:48:12.791Z)

Good! Now let me also check the run_on_vm.sh to see how it handles send_report:

22. user (2026-06-22T09:48:14.026Z)

```
fail_email() {  # best-effort failure alert (the report email won't have fired on an early crash)
  echo "RUN FAILED ? attempting a failure alert"
  if [ "$DO_EMAIL" = "true" ] && [ -d ~/rally ]; then
    cd ~/rally 2>/dev/null && . .venv/bin/activate 2>/dev/null && \
    NIGHTLY_SMTP_SECRET="$SMTP_SECRET" NIGHTLY_SMTP_USER="$SMTP_USER" NIGHTLY_EMAIL_TO="$EMAIL_TO" \
    python - <<PY || true
from scripts.nightly_email import send_report
send_report("[KhelSutra nightly] ERROR ? engine regression VM run failed ($DATE)",
            "The nightly engine-regression run failed on the VM before producing a report.\n"
            "See gs://$BUCKET/$REPORT_PREFIX/$DATE/ and the VM serial log.\nGIT_SHA=$GIT_SHA", to="$EMAIL_TO")
PY
  fi
}
trap 'fail_email' ERR
```

23. assistant (2026-06-22T09:48:17.344Z)

Let me now examine if there are any tests that verify the SMTP error behavior:

24. user (2026-06-22T09:48:18.684Z)

(Bash completed with no output)

25. user (2026-06-22T09:48:21.066Z)

```
tests\test_nightly_regression.py
tests\test_nightly_reelcount.py
```

26. user (2026-06-22T09:48:23.932Z)

```
190:    assert ne.resolve_secret({"NIGHTLY_SMTP_PASS": "hunter2"}) == "hunter2"
199:    assert ne.send_report("s", "b", to="x@y.com", env={"NIGHTLY_EMAIL_TO": "x@y.com"}) is False
```

27. assistant (2026-06-22T09:48:25.923Z)

Let me look at that test file more closely around those lines:

28. user (2026-06-22T09:48:26.975Z)

```
    assert msg["Subject"] == "subj" and msg["To"] == "to@y.com" and msg["From"] == "from@x.com"
    assert "body **md**" in msg.get_content()


def test_email_resolve_secret_from_env():
    assert ne.resolve_secret({"NIGHTLY_SMTP_PASS": "hunter2"}) == "hunter2"


def test_email_resolve_secret_none_when_absent():
    assert ne.resolve_secret({}) is None


def test_email_send_skips_without_creds():
    # No password resolvable ? graceful skip (False), never raises, no network.
    assert ne.send_report("s", "b", to="x@y.com", env={"NIGHTLY_EMAIL_TO": "x@y.com"}) is False


def test_tolerances_default():
    assert nr.Tolerances().quality_tol == 0.02
```

29. assistant (2026-06-22T09:48:32.713Z)

Now I need to analyze the core defect claim. The finding says:
- Lines 79-91 (which I now see are actually 79-91, but the relevant SMTP code is 84-91)
- Claims failed login attempts only log a generic warning
- Claims there's no detailed audit trail

Let me re-examine the actual behavior when SMTP login FAILS (not when the password is missing):

The code flow is:
1. Lines 79-82: If NO password, log warning and return False (this is caught)
2. Lines 84-89: Create SMTP connection, STARTTLS, and LOGIN
   - **NO try-except around lines 84-89**
   - If `s.login()` throws an exception, it will bubble up
   - The exception WILL be caught by the caller (line 539 in nightly_reelcount.py) which logs it generically

Let me verify what exceptions smtplib.login() can raise:

30. user (2026-06-22T09:48:33.805Z)

```
Help on function login in module smtplib:

login(self, user, password, *, initial_response_ok=True)
    Log in on an SMTP server that requires authentication.

    The arguments are:
        - user:         The user name to authenticate with.
        - password:     The password for the authentication.

    Keyword arguments:
        - initial_response_ok: Allow sending the RFC 4954 initial-response
          to the AUTH command, if the authentication methods supports it.

    If there has been no previous EHLO or HELO command this session, this
```

```
    method tries ESMTP EHLO first.

    This method will return normally if the authentication was successful.

    This method may raise the following exceptions:

     SMTPHeloError            The server didn't reply properly to
                              the helo greeting.
     SMTPAuthenticationError  The server didn't accept the username/
                              password combination.
     SMTPNotSupportedError    The AUTH command is not supported by the
                              server.
     SMTPException            No suitable authentication method was
                              found.
```

31. assistant (2026-06-22T09:48:38.229Z)

Excellent. Now I have the full picture. Let me trace what happens on SMTP auth failures:

1. If `s.login(user, password)` raises `SMTPAuthenticationError` (wrong password) or any other exception
2. Lines 86-89 have NO try-except
3. The exception propagates UP to the caller
4. In nightly_reelcount.py lines 534-540, it's caught as `Exception as e` and logged as `log.error("email failed (report still written): %s", e)`

The question is: **Is this actually a defect in production?**

Let me check what information is logged:

32. user (2026-06-22T09:48:41.150Z)

```
Exception str: (535, b'5.7.8 Username and password not accepted')
Exception args: (535, b'5.7.8 Username and password not accepted')
Exception type: SMTPAuthenticationError
```

33. assistant (2026-06-22T09:48:43.151Z)

Now let me verify the actual logging behavior by looking at what gets logged when an exception occurs:

34. user (2026-06-22T09:48:44.262Z)

```
533         if args.email:
534             try:
535                 from scripts.nightly_email import send_report
536                 subject = f"[KhelSutra nightly] {result.overall_status()} ? reel-count engine regression {date}"
537                 sent = send_report(subject, report, to=args.email_to)
538                 log.info("email %s", "sent" if sent else "skipped (no creds ? report still written)")
539             except Exception as e:  # noqa: BLE001 ? a broken mailer must NOT mask the regression verdict
540                 log.error("email failed (report still written): %s", e)
```

35. assistant (2026-06-22T09:48:46.638Z)

Now let me check if there are separate entry points that might NOT have this exception handling (e.g., the shell script):

36. user (2026-06-22T09:48:47.824Z)

```
39      fail_email() {  # best-effort failure alert (the report email won't have fired on an early crash)
```

```
40        echo "RUN FAILED ? attempting a failure alert"
41        if [ "$DO_EMAIL" = "true" ] && [ -d ~/rally ]; then
42          cd ~/rally 2>/dev/null && . .venv/bin/activate 2>/dev/null && \
43          NIGHTLY_SMTP_SECRET="$SMTP_SECRET" NIGHTLY_SMTP_USER="$SMTP_USER"
NIGHTLY_EMAIL_TO="$EMAIL_TO" \
44          python - <<PY || true
45      from scripts.nightly_email import send_report
46      send_report("[KhelSutra nightly] ERROR ? engine regression VM run failed ($DATE)",
47                  "The nightly engine-regression run failed on the VM before producing a
report.\n"
48                  "See gs://$BUCKET/$REPORT_PREFIX/$DATE/ and the VM serial
log.\nGIT_SHA=$GIT_SHA", to="$EMAIL_TO")
49      PY
50        fi
51      }
```

37. assistant (2026-06-22T09:48:52.823Z)

CRITICAL FINDING: Lines 44-49 in run_on_vm.sh call `send_report()` **WITHOUT** any exception handling! The `|| true` only applies to the entire shell command block, not to the Python code. If `send_report()` raises an exception in the Python code (e.g., SMTP authentication failure), it will:

1. Crash the Python inline script
2. The `|| true` suppresses the error from the shell's perspective
3. But the SMTP failure is logged NOWHERE because this is a Python one-liner with no error handling

Let me verify this is actually uncaught:

38. user (2026-06-22T09:48:56.074Z)

```
ERROR: SMTP login failed!
About to call send_report
Exit code: 1
```

39. assistant (2026-06-22T09:49:00.008Z)

Now I see the issue! When `send_report()` raises an exception in the shell's inline Python heredoc, it will:
1. Print to stderr
2. Exit with code 1
3. The `|| true` will suppress that exit code in the shell
4. The failure is silently swallowed

But wait - let me re-read the shell script. The `|| true` is on line 44, which is the Python invocation. Let me trace more carefully:

40. user (2026-06-22T09:49:01.036Z)

```
39      fail_email() {  # best-effort failure alert (the report email won't have fired on an
early crash)
40        echo "RUN FAILED ? attempting a failure alert"
41        if [ "$DO_EMAIL" = "true" ] && [ -d ~/rally ]; then
42          cd ~/rally 2>/dev/null && . .venv/bin/activate 2>/dev/null && \
43          NIGHTLY_SMTP_SECRET="$SMTP_SECRET" NIGHTLY_SMTP_USER="$SMTP_USER"
NIGHTLY_EMAIL_TO="$EMAIL_TO" \
44          python - <<PY || true
45      from scripts.nightly_email import send_report
46      send_report("[KhelSutra nightly] ERROR ? engine regression VM run failed ($DATE)",
47                  "The nightly engine-regression run failed on the VM before producing a
report.\n"
48                  "See gs://$BUCKET/$REPORT_PREFIX/$DATE/ and the VM serial
log.\nGIT_SHA=$GIT_SHA", to="$EMAIL_TO")
```

```
49      PY
50        fi
51      }
52      trap 'fail_email' ERR
53
54      # 1) Observability (Ops Agent) ? same as create_gpu_vm.sh bakes in.
55      curl -sSO https://dl.google.com/cloudagents/add-google-cloud-ops-agent-repo.sh && \
56        sudo bash add-google-cloud-ops-agent-repo.sh --also-install || echo "WARN: ops-agent
install failed (non-fatal)"
57
58      # 2) Pull the repo @ HEAD (staged to GCS by launch.sh ? no GitHub auth needed; uses the
worker SA).
```

41. assistant (2026-06-22T09:49:14.608Z)

The `|| true` is on line 44, but it applies to the entire `python - <<PY ... PY` block. If the Python code raises an unhandled exception, it will:
1. Print the traceback to stderr (which is redirected to `/var/log/nightly-regression.log` by the exec redirect at line 13)
2. Exit with code 1
3. The `|| true` suppresses the exit code

So the error WILL be logged to the VM's system log, but it won't be communicated back in any structured way.

Now let me check the ACTUAL risk: **Can an SMTP authentication failure happen in production?**

Yes, clearly:
1. If the app password is wrong
2. If it's expired
3. If the SMTP server rejects the connection
4. If the credentials are rotated and the secret manager isn't updated

Now I need to determine: **Is the claim in the finding ACCURATE?**

The finding says:
- "Successful SMTP sends are logged (line 90: `log.info('emailed report to %s', to_addr)`)" ? TRUE
- "Failed login attempts only log a generic warning (line 81)" ? MISLEADING/INCOMPLETE
  - Line 81 logs when NO PASSWORD is set, not when login fails
  - When login FAILS (wrong password), the exception is caught at line 539-540 as a generic "email failed"
- "If an attacker exhausts SMTP login retries..." ? UNLIKELY in this context, but theoretically possible
- "no detailed audit trail" ? TRUE for the SMTP authentication step itself
- "credentials are app-password (not the main Gmail password)" ? TRUE
- "account is monitored by the owner" ? TRUE (mitigating factor)

The key defect: When SMTP authentication fails, the error is only logged as a generic Python exception message at line 540 (`"email failed (report still written): %s"` + exception). There is NO detailed logging of:
- Host/port attempted
- Username attempted (though in theory could be inferred)
- The specific SMTP error code
- Number of retry attempts (there are none)

However, this is somewhat mitigated by:
1. The exception IS logged (captured by the except block)
2. The report itself is STILL WRITTEN (never masked)
3. It's a tripwire system (owner-monitored)
4. The failure is captured in the VM log via the ERR trap and shell redirect

Let me now verify if there's any missing logging of AUTH attempt BEFORE the failure:

42. user (2026-06-22T09:49:15.784Z)

```
        return False

    import smtplib
    msg = build_message(subject, body_md, from_addr=user, to_addr=to_addr)
    with smtplib.SMTP(host, port, timeout=60) as s:
        s.starttls()
        s.login(user, password)
        s.send_message(msg)
    log.info("emailed report to %s", to_addr)
    return True
```

43. assistant (2026-06-22T09:49:23.830Z)

There is no logging AT ALL before attempting login. Now, let me assess the severity:

PRODUCTION IMPACT ANALYSIS:

1. **REAL DEFECT?** YES - When SMTP login fails, the logging is inadequate:
   - No attempt logging before `s.login()`
   - No connection/auth details logged on failure
   - Only a generic Python exception string in the catch block
   - SMTP failures in `fail_email()` (lines 46-48) are swallowed by `|| true` with zero logging

2. **SECURITY RISK?** LOW-to-MEDIUM:
   - If an attacker can make credential-stuffing attempts, there's no detailed audit trail
   - The generic exception logging provides minimal forensic value
   - However, the report is NEVER masked (design prevents information loss)
   - Account is app-password only, not the main Gmail password

3. **PRODUCTION SEVERITY:** MEDIUM (not HIGH) because:
   - This is a tripwire/monitoring system (not the main pipeline)
   - Report is never masked (safety-first design)
   - Owner is expected to monitor the system
   - Failure is captured somewhere (VM system log)

RECOMMENDED FIX: Wrap SMTP operations in try-except with detailed logging:
- Log host/port/user before attempting connection
- Log SMTP error code/message on auth failure
- Never log the password
- This especially matters for the `fail_email()` function in run_on_vm.sh

Let me verify the claim one more time by checking if there are any other protections I missed:

44. user (2026-06-22T09:49:25.046Z)

```
1:"""Email the nightly regression report ? stdlib SMTP, creds from env or GCP Secret Manager.
4:dependency-light (stdlib ``smtplib`` + ``email.message``) and **graceful**: if no SMTP
password can
5:be resolved it logs + returns ``False`` (the report is still written/printed ? a broken mailer
must
10:  ? SMTP host:  ``NIGHTLY_SMTP_HOST``  (default ``smtp.gmail.com``)   · port
``NIGHTLY_SMTP_PORT`` (587, STARTTLS)
11:  ? SMTP user:  ``NIGHTLY_SMTP_USER``  (default = recipient)
12:  ? SMTP pass:  ``NIGHTLY_SMTP_PASS``  ?  Secret Manager ``NIGHTLY_SMTP_SECRET``
17:and on the VM:  export NIGHTLY_SMTP_SECRET=projects/<proj>/secrets/nightly-smtp-
pass/versions/latest
18:                export NIGHTLY_SMTP_USER=<your-gmail>   NIGHTLY_EMAIL_TO=avi.dullu@gmail.com
22:import logging
27:log = logging.getLogger("nightly_email")
45:    """Resolve the SMTP password from env or GCP Secret Manager. Returns None if
unresolvable."""
47:    direct = env.get("NIGHTLY_SMTP_PASS")
50:    secret_ref = env.get("NIGHTLY_SMTP_SECRET")
58:    except Exception:  # noqa: BLE001 ? lib absent or auth issue ? try the CLI
67:        log.warning("secret-manager resolution failed: %s", e)
```

```
76:    host = env.get("NIGHTLY_SMTP_HOST", DEFAULT_HOST)
77:    port = int(env.get("NIGHTLY_SMTP_PORT", DEFAULT_PORT))
78:    user = env.get("NIGHTLY_SMTP_USER") or to_addr
81:        log.warning("no SMTP password (set NIGHTLY_SMTP_PASS or NIGHTLY_SMTP_SECRET) ? email skipped.")
86:    with smtplib.SMTP(host, port, timeout=60) as s:
88:        s.login(user, password)
90:    log.info("emailed report to %s", to_addr)
```

45. assistant (2026-06-22T09:49:27.840Z)

Perfect. My analysis is complete. Now let me provide the structured output:

46. user (2026-06-22T09:49:37.680Z)

Structured output provided successfully

47. assistant (2026-06-22T09:49:47.215Z)

Summary

DEFECT CONFIRMED: Real, Medium Severity

The nightly regression engine has **inadequate audit logging for SMTP authentication failures**:

Evidence

1. **scripts/nightly_email.py (lines 84-91)**: The SMTP connection, STARTTLS, and login attempt lack any try-except wrapper. If `s.login(user, password)` fails with an SMTPAuthenticationError (wrong password, expired credentials, server rejection), the exception propagates unhandled.

2. **Exception handling downstream (nightly_reelcount.py:539-540)**: When the exception is caught, it's logged only as a generic string: `"email failed (report still written): %s"` with the Python exception repr. No SMTP-specific details are captured?no error code, no host/port/user logged, no auth attempt logged before the failure.

3. **Critical gap in run_on_vm.sh (lines 45-48)**: The `fail_email()` function calls `send_report()` inside a Python heredoc with `|| true` on the shell command. Any unhandled Python exception is **silently swallowed**?the shell treats it as a non-fatal error and continues. The traceback goes to `/var/log/nightly-regression.log` but with **zero structured audit trail**.

Why It Matters

- **Production risk**: If the SMTP password is wrong, rotated, or the account is compromised, authentication failures are minimally auditable.
- **Mitigating factors**: This is a tripwire/monitoring system (not the main pipeline); the report is never masked; it's app-password only; owner-monitored.
- **Forensic gap**: No connection attempt logging before auth, no SMTP error codes captured, credential-stuffing attempts would leave almost no trace.

Recommended Fix

Wrap lines 86-89 with specific exception handling for SMTP errors; log host/port/username before attempting login; capture SMTP error codes and types separately without logging the password. Additionally, add try-except inside the `fail_email()` Python heredoc in run_on_vm.sh so crash-path email failures are explicitly logged to the startup log.